

SAGI - Manual de Especificaciones Técnicas

Sistema de Administración y Gestión de Inventarios
Instituto de Seguridad Pública (ISeP)

Versión: 1.0
Fecha: 23 de agosto de 2026
Estado: Producción

Índice

- [1. Descripción General](#)
- [2. Arquitectura del Sistema](#)
- [3. Stack Tecnológico](#)
- [4. Modelo de Datos](#)
- [5. Estructura de Base de Datos](#)
- [6. API REST - Endpoints](#)
- [7. Autenticación y Autorización](#)
- [8. Lógica de Negocio](#)
- [9. Frontend - Componentes y Rutas](#)
- [10. Sistema de Auditoría](#)
- [11. Exportación de Datos](#)
- [12. Seguridad](#)
- [13. Requisitos de Instalación](#)

1. Descripción General

SAGI es un sistema web full-stack diseñado para la administración, registro y control de inventarios de bienes muebles del Instituto de Seguridad Pública (ISeP). El sistema permite:

- Registro de ítems con campos dinámicos por categoría
- Control de movimientos (altas, traslados, bajas) con flujo de aprobación
- Gestión de sedes y unidades organizacionales
- Sistema de alertas para pendientes
- Registro de auditoría completo
- Reportes y exportación de datos (Excel, CSV, JSON)

1.1 Alcance

El sistema cubre:

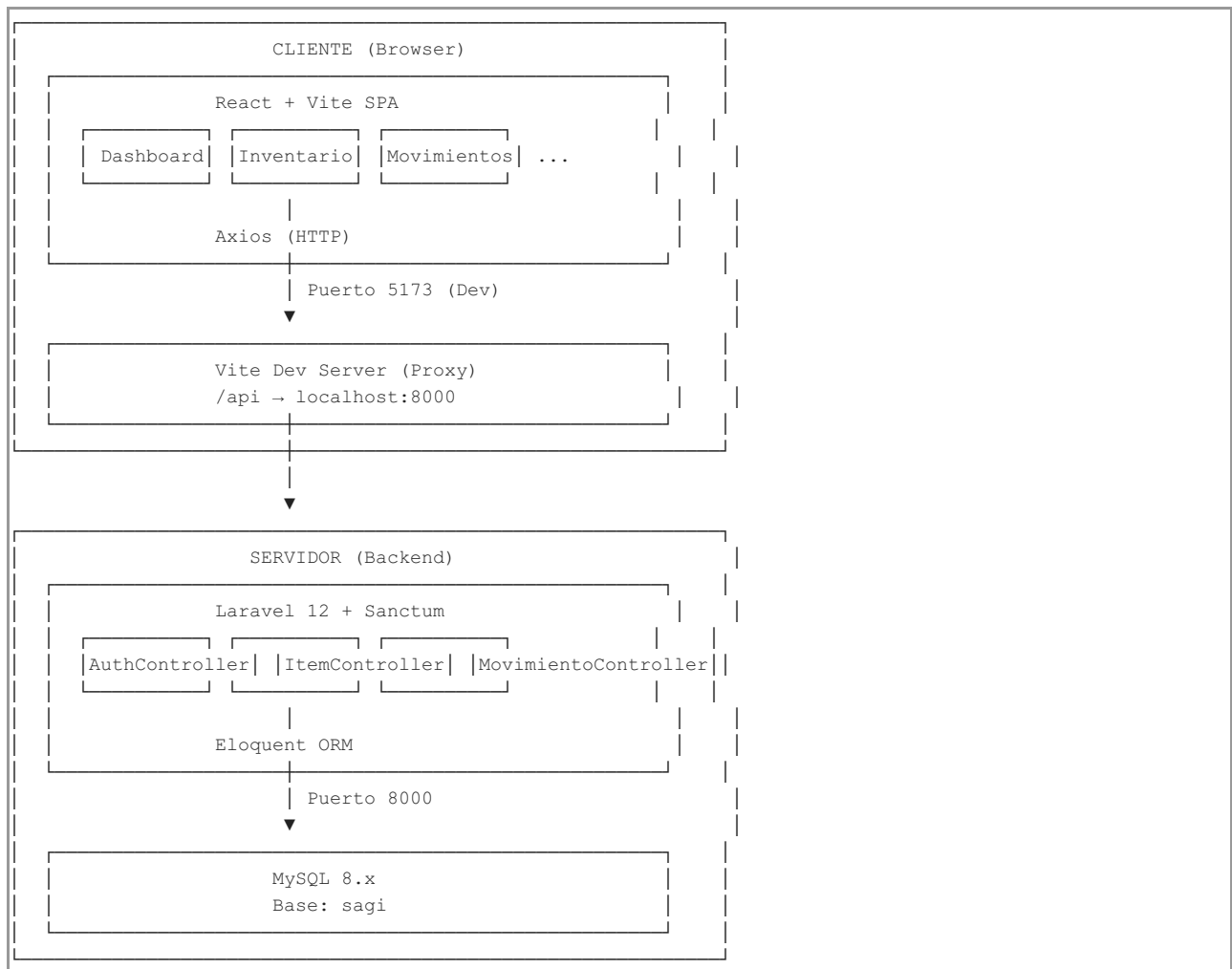
- Inventario de bienes muebles (equipamiento, mobiliario, elementos de oficina, etc.)
- Control de movimientos con aprobación jerárquica
- Trazabilidad completa de cada ítem
- Gestión de categorías con campos personalizables
- Reportes para toma de decisiones

1.2 Público Objetivo

- **Administradores:** Gestión completa del sistema
- **Jefes de Área:** Aprobación de movimientos, reportes
- **Personal de Carga:** Registro de ítems, solicitudes de movimiento
- **Personal de Consulta:** Solo visualización

2. Arquitectura del Sistema

2.1 Arquitectura General



2.2 Flujo de Datos

1. **Autenticación:** El usuario envía DNI + contraseña → Sanctum genera token → token se almacena en localStorage
2. **Peticiones:** Cada petición HTTP incluye header `Authorization: Bearer {token}`
3. **Autorización:** Middleware `rol` verifica el slug del usuario contra los roles permitidos
4. **Respuesta:** JSON estandarizado con datos y/o errores

3. Stack Tecnológico

3.1 Backend

Componente	Versión	Propósito
PHP	8.2+	Lenguaje del servidor
Laravel	12.x	Framework PHP
Sanctum	4.x	Autenticación por tokens
MySQL	8.x	Base de datos relacional
Eloquent ORM	-	Mapeo objeto-relacional

3.2 Frontend

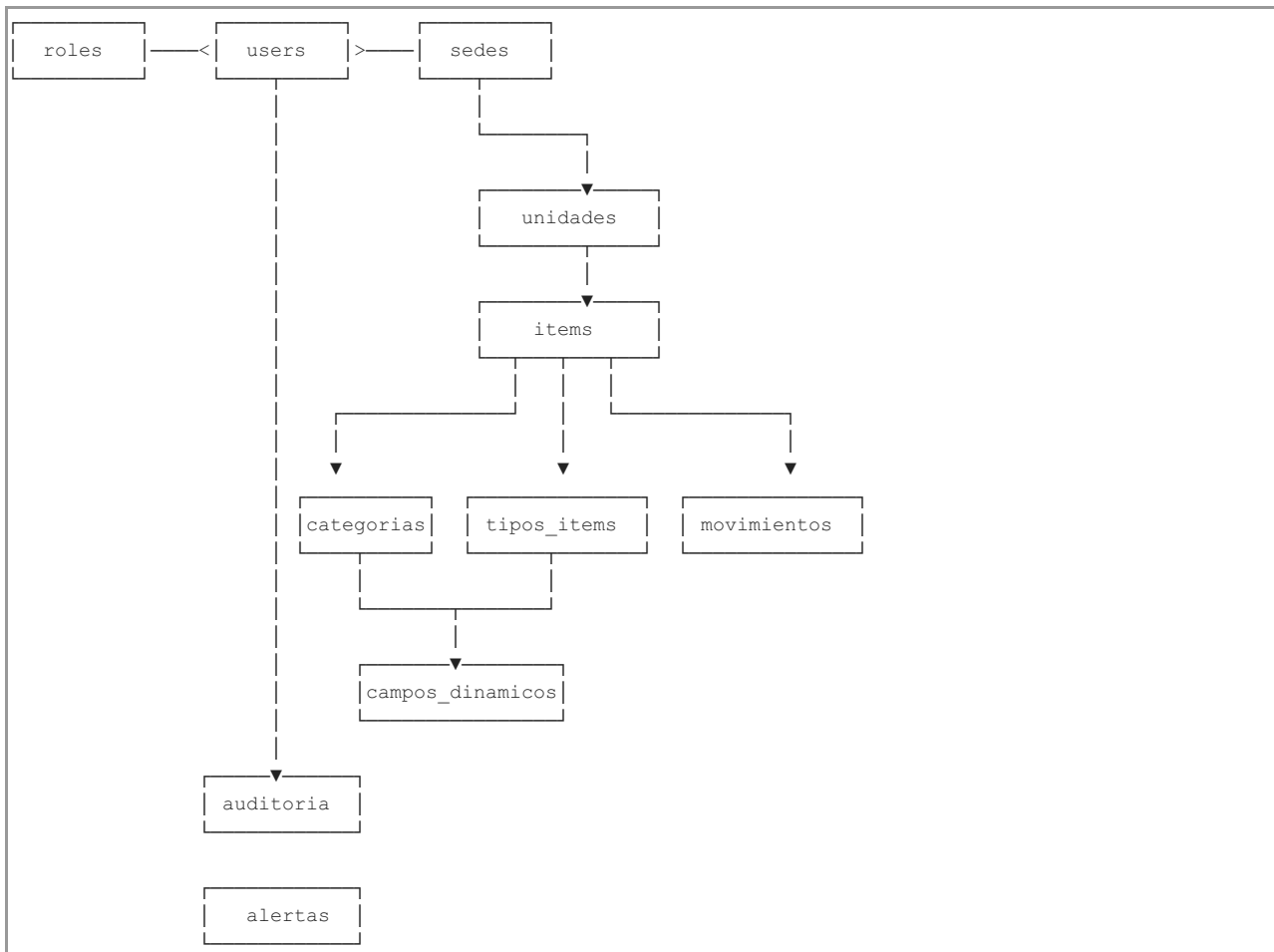
Componente	Versión	Propósito
React	19.x	Biblioteca de UI
Vite	6.x	Bundler y dev server
React Router	7.x	Enrutamiento SPA
Axios	1.x	Cliente HTTP
SheetJS (xlsx)-		Exportación Excel

3.3 Herramientas de Desarrollo

- **Git:** Control de versiones (GitHub: <https://github.com/lolanga/sagi.git>)
- **Node.js:** Runtime de JavaScript
- **Composer:** Gestor de dependencias PHP

4. Modelo de Datos

4.1 Diagrama de Relaciones



4.2 Entidades Principales

Roles

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
nombre	VARCHAR(50)	Nombre descriptivo
slug	VARCHAR(50) (UNIQUE)	Identificador técnico

Valores posibles: admin, jefe, carga, consulta

Users (Usuarios)

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
name	VARCHAR(255)	Nombre completo
email	VARCHAR(255) (UNIQUE)	Correo electrónico
dni	VARCHAR(8) (UNIQUE)	Documento Nacional de Identidad
username	VARCHAR(50) (UNIQUE)	Nombre de usuario
rol_id	INT (FK → roles)	Rol asignado
sede_id	INT (FK → sedes)	Sede de trabajo
password	VARCHAR(255)	Contraseña hasheada (bcrypt)

Sedes

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
nombre	VARCHAR(255)	Nombre de la sede
activa	BOOLEAN	Estado (true=activa, false=inactiva)

Unidades

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
sede_id	INT (FK → sedes)	Sede a la que pertenece
nombre	VARCHAR(255)	Nombre de la unidad
activa	BOOLEAN	Estado (true=activa, false=inactiva)

Categorías

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
codigo	VARCHAR(4) (UNIQUE)	Código abreviado (ej: A1, A2)
nombre	VARCHAR(255)	Nombre descriptivo
es_transitoria	BOOLEAN	Si es categoría temporal (A7=alta, A8=baja)

Categorías predefinidas:

- A1: Equipamiento Tecnológico
- A2: Mobiliario de Oficina
- A3: Elementos de Seguridad
- A4: Vehículos
- A5: Herramientas
- A6: Material Bibliográfico
- A7: Categoría Temporal de Alta (transitoria)
- A8: Categoría de Baja (transitoria)

Tipos de Item (TiposItems)

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
categoria_id	INT (FK → categorias)	Categoría padre
nombre	VARCHAR(255)	Nombre del tipo
orden	INT	Posición en la lista

Campos Dinámicos

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
categoria_id	INT (FK → categorias)	Categoría padre
tipo_item_id	INT (FK → tipos_items, nullable)	Tipo específico (null=general)
nombre	VARCHAR(255)	Nombre del campo
tipo	ENUM	tipo, numero, fecha, select, textarea
opciones	JSON	Opciones para tipo "select"
placeholder	VARCHAR(255)	Texto de ayuda
requerido	BOOLEAN	Si es obligatorio
activo	BOOLEAN	Si está habilitado
orden	INT	Posición en la lista

Tipos de campo disponibles:

- texto: Campo de texto corto (input)
- numero: Campo numérico (input number)
- fecha: Campo de fecha (input date)
- select: Lista desplegable (opciones en JSON)
- textarea: Campo de texto largo

Items (Inventario)

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
codigo_unico	VARCHAR(20) (UNIQUE)	Código del ítem (ej: A1-03-47-000001)
categoria_id	INT (FK → categorias)	Categoría
tipo_item_id	INT (FK → tipos_items, nullable)	Tipo específico
responsable_id	INT (FK → users)	Persona responsable
unidad_id	INT (FK → unidades)	Unidad de destino
estado_conservacion	VARCHAR(50)	Estado físico
cantidad	INT	Cantidad (default: 1)
fecha_alta	DATE (nullable)	Fecha de incorporación
valores_dinamicos	JSON	Valores de campos dinámicos
estado	ENUM	activo, pendiente, baja

Formato del código único: {Categoría}-{IdSede 2dígitos}-{IdUnidad 2dígitos}-{Secuencial 6dígitos}
Ejemplo: A1-03-47-000001

Estados de conservación: Muy bueno, Bueno, Regular, Malo

Movimientos

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
item_id	INT (FK → items)	Ítem afectado
tipo	ENUM	alta, traslado, baja
unidad_origen_id	INT (FK → unidades)	Unidad de origen
unidad_destino_id	INT (FK → unidades, nullable)	Unidad de destino
motivo	TEXT	Razón del movimiento
estado	ENUM	pendiente, aprobado, rechazado
solicitante_id	INT (FK → users)	Quien solicita
validador_id	INT (FK → users, nullable)	Quien aprueba/rechaza
fecha_validacion	DATETIME	Fecha de validación
motivo_rechazo	TEXT	Razón del rechazo

Alertas

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
tipo	ENUM	pendiente_aprobacion, pendiente_movimiento, manual
prioridad	ENUM	critica, importante, informativa
estado	ENUM	abierta, cerrada
item_id	INT (FK → items, nullable)	Ítem relacionado
movimiento_id	INT (FK → movimientos, nullable)	Movimiento relacionado
unidad_id	INT (FK → unidades)	Unidad afectada
mensaje	TEXT	Descripción de la alerta
fecha_cierre	DATETIME	Fecha de cierre

Auditoría

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
user_id	INT (FK → users)	Usuario que realizó la acción
accion	VARCHAR(50)	Tipo de acción
entidad	VARCHAR(50)	Entidad afectada
entidad_id	INT (nullable)	ID de la entidad
detalle	JSON	Detalle adicional

Acciones registradas: crear, editar, eliminar, aprobar, rechazar, login, solicitar, mover, cerrar, activar, desactivar

5. Estructura de Base de Datos

5.1 Nombre de la Base de Datos

sagi

5.2 Conexión

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=sagi
DB_USERNAME=root
DB_PASSWORD=
```

5.3 Migraciones

El sistema utiliza 19 migraciones para crear y modificar las tablas:

Migración	Tabla	Acción
000000	roles	Crear tabla de roles
000000	sedes	Crear tabla de sedes
000000	unidades	Crear tabla de unidades
000000	users	Crear tabla de usuarios
000001	cache	Crear tablas de caché
000002	jobs	Crear tablas de trabajos
000003	categorias	Crear tabla de categorías
000004	campos_dinamicos	Crear tabla de campos dinámicos
000005	items	Crear tabla de ítems
000006	movimientos	Crear tabla de movimientos
000007	alertas	Crear tabla de alertas
000008	auditoria	Crear tabla de auditoría
000009	tipos_items	Crear tabla de tipos de ítem
000010	items	Agregar tipo_item_id
000011	campos_dinamicos	Agregar tipo_item_id y placeholder
20260818	personal_access_tokens	Crear tabla de tokens Sanctum
20260818	sedes, unidades	Agregar campo activa
20260823	items	Hacer fecha_alta nullable
20260823	items	Recodificar códigos existentes

6. API REST - Endpoints

6.1 Autenticación

Método	Endpoint	Descripción	Autenticación	Roles
POST	/api/login	Iniciar sesión	No	Público
GET	/api/me	Obtener usuario actual	Sí	Todos
POST	/api/logout	Cerrar sesión	Sí	Todos

6.2 Dashboard

Método	Endpoint	Descripción	Autenticación	Roles
GET	/api/dashboard/stats	Estadísticas generales	Sí	Todos

6.3 Ítems (Inventario)

Método	Endpoint	Descripción	Autenticación	Roles
GET	/api/items	Listar ítems (paginado, 20/pág)	Sí	Todos
POST	/api/items	Crear ítem (alta)	Sí	admin, jefe, carga
GET	/api/items/{id}	Ver detalle de ítem	Sí	Todos
PUT	/api/items/{id}	Actualizar ítem	Sí	admin, jefe, carga
DELETE	/api/items/{id}	Eliminar ítem	Sí	admin, jefe, carga

Parámetros de filtro para GET /api/items:

- search: Búsqueda por código o valores dinámicos
- categoria_id: Filtrar por categoría
- estado_conservacion: Filtrar por estado de conservación
- estado: Filtrar por estado (activo/pendiente/baja)
- unidad_id: Filtrar por unidad

- page: Número de página (default: 1)

6.4 Categorías

Método	Endpoint	Descripción	Autenticación	Roles
GET	/api/categorias	Listar categorías	Sí	Todos
POST	/api/categorias	Crear categoría	Sí	admin
PUT	/api/categorias/{id}	Actualizar categoría	Sí	admin
DELETE	/api/categorias/{id}	Eliminar categoría	Sí	admin
GET	/api/categorias/{id}/campos	Obtener campos dinámicos	Sí	Todos
POST	/api/categorias/{id}/campos	Crear campo dinámico	Sí	admin
GET	/api/categorias/{id}/tipos	Obtener tipos de ítem	Sí	Todos
POST	/api/categorias/{id}/tipos	Crear tipo de ítem	Sí	admin

6.5 Campos Dinámicos

Método	Endpoint	Descripción	Autenticación	Roles
PUT	/api/campos-dinamicos/{id}	Actualizar campo	Sí	admin
DELETE	/api/campos-dinamicos/{id}	Eliminar campo	Sí	admin
POST	/api/campos-dinamicos/{id}/mover	Reordenar campo	Sí	admin

6.6 Tipos de Ítem

Método	Endpoint	Descripción	Autenticación	Roles
PUT	/api/tipos-item/{id}	Actualizar tipo	Sí	admin
DELETE	/api/tipos-item/{id}	Eliminar tipo	Sí	admin
POST	/api/tipos-item/{id}/mover	Reordenar tipo	Sí	admin

6.7 Sedes

Método	Endpoint	Descripción	Autenticación	Roles
GET	/api/sedes	Listar sedes	Sí	Todos
POST	/api/sedes	Crear sede	Sí	admin
PUT	/api/sedes/{id}	Actualizar sede	Sí	admin
DELETE	/api/sedes/{id}	Eliminar sede	Sí	admin

6.8 Unidades

Método	Endpoint	Descripción	Autenticación	Roles
GET	/api/unidades	Listar unidades	Sí	Todos
POST	/api/unidades	Crear unidad	Sí	admin
PUT	/api/unidades/{id}	Actualizar unidad	Sí	admin
DELETE	/api/unidades/{id}	Eliminar unidad	Sí	admin

6.9 Movimientos

Método	Endpoint	Descripción	Autenticación	Roles
GET	/api/movimientos	Listar movimientos (paginado, 25/pág)	Sí	Todos
POST	/api/movimientos/traslados	Solicitar traslado	Sí	admin, jefe, carga
POST	/api/movimientos/bajas	Solicitar baja	Sí	admin, jefe, carga
POST	/api/movimientos/{id}/aprobar	Aprobar movimiento	Sí	admin, jefe
POST	/api/movimientos/{id}/rechazar	Rechazar movimiento	Sí	admin, jefe

Parámetros de filtro para GET /api/movimientos:

- tipo: Filtrar por tipo (alta/traslado/baja)
- estado: Filtrar por estado (pendiente/aprobado/rechazado)
- item_id: Filtrar por ítem

6.10 Alertas

Método	Endpoint	Descripción	Autenticación	Roles
GET	/api/alertas	Listar alertas (paginado, 50/pág)	Sí	Todos
POST	/api/alertas	Crear alerta manual	Sí	admin, jefe
POST	/api/alertas/{id}/cerrar	Cerrar alerta	Sí	admin, jefe

6.11 Auditoría

Método	Endpoint	Descripción	Autenticación	Roles
GET	/api/auditoria	Listar registros (paginado, 50/pág)	Sí	admin, jefe

Parámetros de filtro para GET /api/auditoria:

- entidad: Filtrar por entidad
- accion: Filtrar por acción
- user_id: Filtrar por usuario
- desde: Fecha inicio (YYYY-MM-DD)
- hasta: Fecha fin (YYYY-MM-DD)

6.12 Reportes

Método	Endpoint	Descripción	Autenticación	Roles
GET	/api/reportes/resumen	Datos agregados	Sí	admin, jefe
GET	/api/reportes/items	Lista completa de ítems	Sí	admin, jefe

7. Autenticación y Autorización

7.1 Autenticación (Laravel Sanctum)

El sistema utiliza **Laravel Sanctum** para la autenticación basada en tokens.

Flujo de login:

- El cliente envía POST /api/login con { dni, password }
- El servidor valida credenciales contra la tabla users
- Se genera un token de acceso (Personal Access Token)
- Se retorna { token, user: { id, name, dni, username, rol, sede } }
- El cliente almacena el token en localStorage bajo la clave sagi_token
- Cada petición subsiguiente incluye Authorization: Bearer {token}

Cierre de sesión:

- El cliente envía POST /api/logout
- El servidor revoca el token actual
- El cliente limpia localStorage

7.2 Autorización (Roles)

El sistema implementa control de acceso basado en roles (RBAC) mediante el middleware rol.

Roles Definidos

Rol	Slug	Descripción	Permisos
Administrador	admin	Acceso total	CRUD completo sobre todas las entidades
Jefe de Área	jefe	Gestión intermedia	Ver todo, aprobar/rechazar movimientos, crear/cerrar alertas
Personal de Carga	carga	Registro de datos	Crear ítems, solicitar movimientos
Personal de Consulta	consulta	Solo lectura	Ver dashboard, inventario, movimientos, reportes

Matriz de Permisos

Funcionalidad	admin	jefe	carga	consulta
Dashboard	✓	✓	✓	✓
Ver Inventario	✓	✓	✓	✓
Crear Ítem	✓	✓	✓	X
Editar Ítem	✓	✓	✓	X
Eliminar Ítem	✓	✓	✓	X
Ver Detalle Ítem	✓	✓	✓	✓
Ver Movimientos	✓	✓	✓	✓
Solicitar Traslado	✓	✓	✓	X
Solicitar Baja	✓	✓	✓	X
Aprobar/Rechazar	✓	✓	X	X
Ver Reportes	✓	✓	X	X

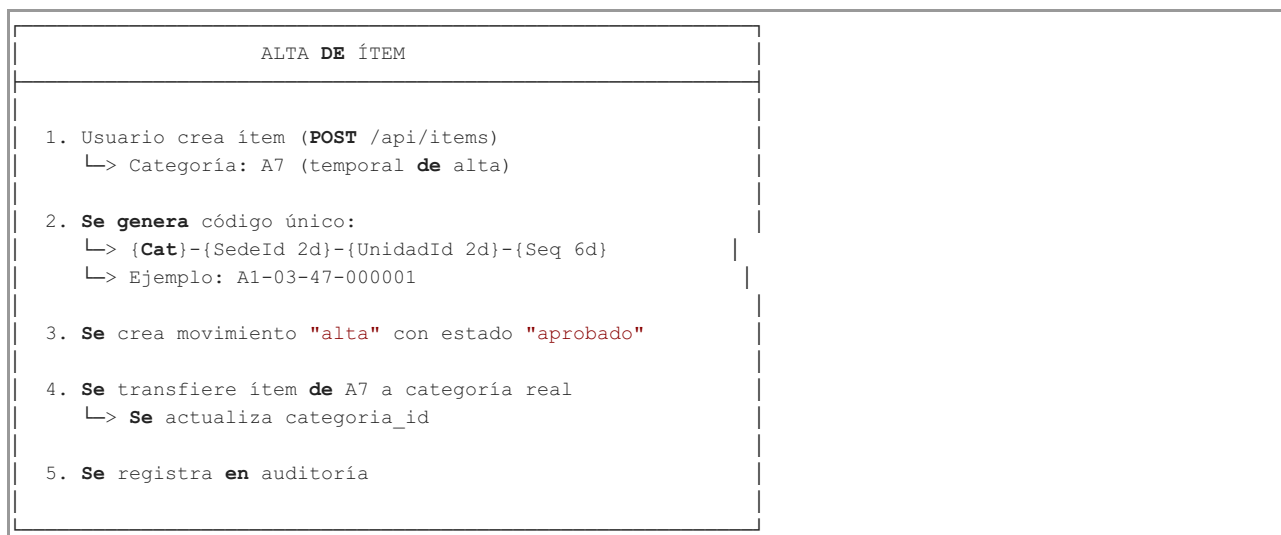
Exportar Reportes	✓	✓	X	X
Ver Alertas	✓	✓	✓	✓
Crear Alerta	✓	✓	X	X
Cerrar Alerta	✓	✓	X	X
Gestionar Categorías	✓	X	X	X
Gestionar Sedes	✓	X	X	X
Gestionar Unidades	✓	X	X	X
Ver Auditoría	✓	✓	X	X
Exportar Auditoría	✓	✓	X	X

7.3 Usuarios de Prueba

DNI	Contraseña	Rol	Usuario
10000001	Admin1234	admin	admin
10000002	Jefe1234	jefe	jefe
10000003	Carga1234	carga	carga
10000004	Consulta1234	consulta	consulta

8. Lógica de Negocio

8.1 Flujo de Alta de Ítem



8.2 Flujo de Traslado



8.3 Flujo de Baja

BAJA DE ÍTEM

1. Usuario solicita baja (**POST** /api/movimientos/bajas)
2. **Se** crea movimiento con estado "pendiente"
3. **Se** crea alerta "crítica"
4. Admin/Jefe aprueba:
 - └> **Se** cambia estado del ítem a "baja"
 - └> **Se** cambia categoría a A8 (baja)
5. **Se** cierra **la** alerta asociada
6. **Se** registra **en** auditoría

8.4 Generación de Código Único

El código único de cada ítem sigue el formato:

{Categoría}-{IdSede}-{IdUnidad}-{Secuencial}

Ejemplo: A1-03-47-000001

- A1: Código de categoría (Equipamiento Tecnológico)
- 03: ID de sede (2 dígitos)
- 47: ID de unidad (2 dígitos)
- 000001: Secuencial único (6 dígitos)

Algoritmo de generación:

1. Obtener código de categoría
2. Obtener ID de sede (formateado a 2 dígitos)
3. Obtener ID de unidad (formateado a 2 dígitos)
4. Buscar el último secuencial para esa combinación
5. Incrementar en 1 y formatear a 6 dígitos

8.5 Sistema de Campos Dinámicos

Cada categoría puede tener campos personalizados que se aplican a sus ítems:

- **Campos generales:** Aplican a todos los ítems de la categoría
- **Campos por tipo:** Aplican solo a un tipo específico de ítem

Tipos de campo disponibles:

- texto: Input de texto corto
- numero: Input numérico
- fecha: Selector de fecha
- select: Lista desplegable (opciones en JSON)
- textarea: Texto largo

Los valores se almacenan en la columna `valores_dinamicos` (JSON) del ítem.

9. Frontend - Componentes y Rutas

9.1 Estructura de Directorios

```

frontend/src/
├── components/
│   ├── AuthContext.jsx      # Contexto de autenticación
│   ├── Layout.jsx          # Shell de la aplicación
│   ├── Modal.jsx           # Componente modal genérico
│   ├── Aviso.jsx           # Banner de notificación
│   ├── ItemForm.jsx        # Formulario de ítem
│   └── ItemDetalle.jsx     # Vista de detalle de ítem
├── pages/
│   ├── Login.jsx           # Página de login
│   ├── Dashboard.jsx       # Panel principal
│   ├── Inventario.jsx      # Lista de inventario
│   ├── Movimientos.jsx     # Gestión de movimientos
│   ├── Categorías.jsx      # Gestión de categorías
│   ├── Unidades.jsx        # Gestión de sedes/unidades
│   ├── Alertas.jsx         # Gestión de alertas
│   ├── Auditoria.jsx       # Registro de auditoría
│   └── Reportes.jsx        # Reportes y exportación
├── services/
│   └── api.js              # Instancia Axios configurada
├── styles/
│   ├── auth.css            # Estilos de login
│   ├── dashboard.css       # Estilos de dashboard/reportes
│   ├── inventario.css      # Estilos de inventario/movimientos
│   ├── categorías.css      # Estilos de categorías
│   ├── form.css            # Estilos de formularios
│   ├── modal.css           # Estilos de modales
│   ├── aviso.css           # Estilos de avisos
│   ├── detalle.css         # Estilos de detalle
│   └── index.css           # Estilos globales
└── index.css               # Estilos globales

```

9.2 Rutas

Ruta	Componente	Protección	Roles Permitidos
/login	Login	Pública	Todos
/	Dashboard	ProtectedRoute	Todos autenticados
/inventario	Inventario	ProtectedRoute	Todos autenticados
/categorias	Categorías	RoleRoute	admin
/unidades	Unidades	RoleRoute	admin
/movimientos	Movimientos	ProtectedRoute	admin, jefe, carga
/reportes	Reportes	ProtectedRoute	admin, jefe
/alertas	Alertas	ProtectedRoute	admin, jefe, carga
/auditoria	Auditoria	ProtectedRoute	admin, jefe

9.3 Componentes Principales

Layout.jsx

- Shell de la aplicación con sidebar, topbar y área de contenido
- Sidebar responsive con navegación filtrada por rol
- Topbar con información de usuario y acciones
- Menú hamburguesa para móvil (< 900px)

Modal.jsx

- Diálogo modal genérico
- Props: open, title, onClose, children, wide
- Cierra con Escape y clic fuera
- Bloquea scroll del body

ItemForm.jsx

- Formulario completo para crear/editar ítems
- Carga dinámica de campos según categoría y tipo seleccionado
- Validación de campos requeridos
- Soporte para fecha desconocida

ItemDetalle.jsx

- Vista de solo lectura del ítem
- Muestra datos generales y campos dinámicos
- Historial de movimientos asociados

9.4 Servicios (API)

El archivo `frontend/src/services/api.js` configura una instancia de Axios con:

- **Base URL:** `/api` (proxy a `localhost:8000`)
- **Request Interceptor:** Agrega header `Authorization: Bearer {token}`
- **Response Interceptor:** En caso de 401, limpia sesión y redirige a `/login`

10. Sistema de Auditoría

10.1 Acciones Registradas

Acción	Descripción
<code>login</code>	Inicio de sesión
<code>crear</code>	Creación de entidad
<code>editar</code>	Edición de entidad
<code>eliminar</code>	Eliminación de entidad
<code>aprobar</code>	Aprobación de movimiento
<code>rechazar</code>	Rechazo de movimiento
<code>solicitar</code>	Solicitud de movimiento
<code>mover</code>	Reordenamiento de elementos
<code>cerrar</code>	Cierre de alerta
<code>activar</code>	Activación de sede/unidad
<code>desactivar</code>	Desactivación de sede/unidad

10.2 Entidades Auditadas

- Usuarios (`login`)
- Items (`crear`, `editar`, `eliminar`)
- Categorías (`crear`, `editar`, `eliminar`)
- Campos Dinámicos (`crear`, `editar`, `eliminar`, `mover`)
- Tipos de Ítem (`crear`, `editar`, `eliminar`, `mover`)
- Sedes (`crear`, `editar`, `eliminar`, `activar`, `desactivar`)
- Unidades (`crear`, `editar`, `eliminar`, `activar`, `desactivar`)
- Movimientos (`solicitar`, `aprobar`, `rechazar`)
- Alertas (`crear`, `cerrar`)

10.3 Estructura del Registro

Cada registro de auditoría contiene:

- `user_id`: Usuario que realizó la acción
- `accion`: Tipo de acción (`crear`/`editar`/`eliminar`/etc.)
- `entidad`: Nombre de la entidad afectada
- `entidad_id`: ID de la entidad (nullable)
- `detalle`: JSON con información adicional (valores anteriores/nuevos, IP, etc.)

11. Exportación de Datos

11.1 Reportes

El módulo de reportes permite exportar:

- **Resumen por categoría:** Código, nombre, cantidad de ítems
- **Resumen por estado de conservación:** Estado, cantidad
- **Resumen por sede:** Sede, cantidad
- **Resumen por unidad:** Unidad, cantidad
- **Top 20 elementos:** Elemento más frecuente
- **Movimientos por mes:** Últimos 6 meses

11.2 Formatos de Exportación

Formato	Uso	Descripción
Excel (.xlsx)	Reportes	Una hoja por categoría con campos dinámicos
CSV	Reportes	Separado por comas
JSON	Auditoría	Formato estructurado

11.3 Exportación de Auditoría

- **Excel:** Columnas [Fecha, Usuario, Acción, Entidad, Detalle]
- **JSON:** Registros completos con toda la información

12. Seguridad

12.1 Autenticación

- Contraseñas hasheadas con bcrypt
- Tokens de acceso con Sanctum (Personal Access Tokens)
- Tokens almacenados en localStorage (no en cookies)
- Revocación de tokens al cerrar sesión

12.2 Autorización

- Middleware `rol` verifica roles permitidos por endpoint
- Control de acceso basado en roles (RBAC)
- Validación de permisos en frontend (RoleRoute)

12.3 Validación

- Validación de datos en todos los endpoints de escritura
- Reglas de unicidad case-insensitive (sedes, unidades, categorías)
- Validación de campos dinámicos según categoría
- Protección contra inyección SQL (Eloquent ORM)

12.4 Protección de Datos

- Auditoría completa de acciones
- Eliminación segura de ítems (limpieza de alertas y movimientos)
- Baja lógica de sedes y unidades (campo `activa`)
- Estados de ítem: activo, pendiente, baja

13. Requisitos de Instalación

13.1 Requisitos del Servidor

- PHP 8.2 o superior
- MySQL 8.x
- Composer
- Node.js 18+ y npm
- XAMPP (recomendado para desarrollo local)

13.2 Instalación del Backend

```
cd backend
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate
php artisan db:seed
php artisan serve --port=8000
```

13.3 Instalación del Frontend

```
cd frontend
npm install
npm run dev
```

13.4 Variables de Entorno

```
APP_NAME=SAGI
APP_ENV=local
APP_KEY=base64:...
APP_DEBUG=true
APP_URL=http://localhost:5173

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=sagi
DB_USERNAME=root
DB_PASSWORD=

SANCTUM_STATEFUL_DOMAINS=localhost:5173
```

Apéndice A: Formato del Código Único

```
A1-03-47-000001
|  |  |  |
|  |  |  |  Secuencial (6 dígitos)
|  |  |  |  ID Unidad (2 dígitos)
|  |  |  |  ID Sede (2 dígitos)
|  |  |  |  Código Categoría (2 caracteres)
```

Apéndice B: Estados del Sistema

Estados de Ítem

- activo: Ítem en uso normal
- pendiente: Ítem esperando aprobación
- baja: Ítem dado de baja

Estados de Movimiento

- pendiente: Esperando aprobación
- aprobado: Movimiento ejecutado
- rechazado: Movimiento denegado

Estados de Alerta

- abierta: Alerta activa
- cerrada: Alerta resuelta

Estados de Conservación

- Muy bueno: Como nuevo
- Bueno: Uso normal, sin daños
- Regular: Desgaste moderado
- Malo: Requiere reparación/reemplazo